

Airtable to Salesforce: what loaded, what didn't, and why

A reconciliation of every Airtable table against the records now in the Salesforce sandbox, following a complete wipe and reload that proved the loading process end to end — plus the transformations applied along the way and the decisions still needed from the Partnerships team.

Environment Dev sandbox (peodv8dvn) Report date 13 August 2026, after the rebuild
Source Airtable, pulled 12 August 2026 Supersedes the earlier 13 August report

Where things stand

The Salesforce sandbox was **completely emptied and rebuilt from scratch** on 13 August, to prove the loading process end to end rather than only its results. Everything below was loaded by that single rehearsed run. Two data sets that had been built but never actually loaded — notes, and the links between related applications — are now in place. Meetings remains deliberately not started.

6,879 Records created or updated in Salesforce	537 Notes migrated — the last remaining data set	166 Unmatched accounts — still the largest blocker	1,845 Meetings not yet started
--	--	--	--

THE ONE NUMBER TO WATCH

A single issue — **166 Airtable account rows that don't match an account already in Salesforce** — is responsible for the majority of every other gap on this page. It blocks partner accounts, which blocks applications, which blocks the application–contact links and 200 of the notes. Resolving those 166 rows would pull several hundred additional records into Salesforce with no further engineering work.

WHY THE SANDBOX WAS EMPTIED FIRST

Record ownership — who each record belongs to — is decided at the moment a record is loaded. It could not be checked without deleting everything and loading it again. Doing so was worth it: the rehearsal uncovered **six defects that no amount of review would have found**, including one that would have silently mis-assigned 150 applications and another that would have left 100 notes stranded and untraceable. All six are fixed. The process is now materially more trustworthy than it was before the rehearsal, which was the point.

Record reconciliation

Airtable row counts against what is now in Salesforce. A negative difference is not necessarily data loss — in several cases it is intentional consolidation, explained in the final column.

DATA SET	AIRTABLE	SALESFORCE	DIFF	%	STATUS	EXPLANATION
Market Segments	6	6	0	100%	COMPLETE	Loaded previously; untouched by this migration.
Accounts	757	583	-174	77.0%	NEEDS DECISIONS	Accounts already existed in Salesforce, so these were matched rather than created. 169 Airtable rows matched nothing. Spot-checking found most are duplicate rows <i>within Airtable</i> — e.g. "Army" alongside "Department of the Army".
Partner Accounts	99	74	-25	74.7%	NEEDS DECISIONS	20 are blocked by the unmatched accounts above. 5 have no usable account link at all (4 marked inactive, 1 linked to two accounts).

DATA SET	AIRTABLE	SALESFORCE	DIFF	%	STATUS	EXPLANATION
Contacts	2,119	1,548	-571	73.1%	LOADED	Source is two tables: 1,599 Contacts + 520 Opportunity Contacts. The difference is mostly <i>intentional</i> — the same person appearing on several rows was consolidated into one contact. 45 rows had neither a name nor an email and could not be loaded; 8 were rejected as duplicates of existing Salesforce contacts.
Opportunities	928	742	-186	80.0%	NEEDS DECISIONS	142 blocked by unmatched accounts, 28 have no status recorded, 16 have no account link.
Applications	1,064	688	-376	64.7%	NEEDS DECISIONS	359 are blocked because their partner account hasn't loaded — again tracing back to the unmatched accounts. 17 have no partner account link (6 retired applications, 11 in-progress drafts).
Impediments	41	39	-2	95.1%	LOADED	2 rows are entirely blank and look like accidental empty records.
Application ↔ Contact links	2,797	1,878	-919	67.1%	PARTIAL	A link can only exist once both sides do; 849 are waiting on an application that hasn't loaded. 33 were duplicates created by consolidating contacts and were merged.

DATA SET	AIRTABLE	SALESFORCE	DIFF	%	STATUS	EXPLANATION
Impediment ↔ Opportunity links	783	267	-516	34.1%	NEEDS DECISIONS	465 of these point at a single impediment named “None”, which we believe is a placeholder meaning <i>no impediment</i> — see below. 44 more await an opportunity that hasn't loaded.
Notes	796	537	-259	67.5%	LOADED	New in this run. Free-text Airtable columns with no dedicated Salesforce field — application notes, launch notes, partner account descriptions and similar — migrated as notes attached to the relevant record. 200 await a parent record that hasn't loaded; 59 were skipped because their content was literally “None” or “N/A”.
Related application links	70	63	-7	90.0%	LOADED	New in this run. Links between broker applications and their parent application. 6 await an application that hasn't loaded; 1 lists itself as its own parent and was dropped.
Opportunity contact roles	520	516	-4	99.2%	LOADED	437 of 520 source rows loaded, producing 515 role records because some people hold more than one role. 83 rows await an opportunity or contact.
Meetings	1,845	0	-1,845	0%	NOT STARTED	Deferred — Airtable holds no meeting times. See “Known issue: meetings cannot be migrated as calendar events”.

The contacts finding

This is the most consequential thing we learned about the source data, and it changes how contacts should be maintained in Airtable going forward.

Most contacts have no name recorded — and this was hidden until the rebuild

970 of the 1,548 migrated contacts (62%) have no name in Airtable and were loaded with their email address in the surname field, because Salesforce requires a surname and an email is better than a placeholder.

This figure was previously invisible, and the reason is worth understanding. Each time the load ran, it checked Salesforce for a contact with a matching email and reused the name it found — sensible behaviour for recovering a name someone had since corrected by hand. But once the migration had run at least once, what it found was *its own earlier output*. The report read “970 names recovered from Salesforce”, which sounded like a success and was in fact the process reading back its own placeholders. Emptying the sandbox removed that echo and the real number appeared.

WHAT THIS MEANS FOR YOU

These 970 contacts will appear in Salesforce listed by email address rather than by name. They are correct, findable and linked to the right agencies — but they will not read well in a contact list. Adding names in Airtable for the ones that matter, before go-live, is the only fix; the migration cannot invent them. A re-run picks up whatever has been filled in.

The contacts attached to opportunities are a separate population

Airtable has a **Contacts** table (1,599 rows) and, separately, an **Opportunity Contacts** table (520 rows). We expected the second to reference the first. It does not — it has no link to the Contacts table at all, only a typed name and an email address.

When we checked those 520 people against the Contacts table:

- **65** existed in the Contacts table.
- **348** did not appear there at all.
- **107** had no email address recorded, only a name.

In other words, roughly two-thirds of the people attached to opportunities existed nowhere else in the base. They had to be created as contacts in their own right before their roles on opportunities could be recorded — a separate load, not a lookup.

WHY THIS MATTERS BEYOND THE MIGRATION

Salesforce treats a person as one record who can be attached to many opportunities and applications. Airtable currently has no equivalent link, so the same person gets typed in again for each thing they're involved with. That's why the Contacts table also contains duplicates: **61 email addresses appear on two or more rows**, one of them four times. We consolidated those into single contacts on load, but the underlying pattern will keep producing duplicates until Airtable has a proper way to link one person to several applications.

A related case: the same person, several roles

62 of the opportunity contact rows list more than one role for the same person on the same opportunity — for example *Senior POC* + *Decision Maker* + *Day-to-Day POC*. Salesforce stores one role per record, so this needed an explicit decision rather than a default.

Decision: create one record per role

The alternative was to pick a single “most senior” role and discard the others, which would have silently lost information the team had deliberately recorded.

Salesforce permits the same contact to hold several roles on one opportunity, so all of them are preserved. Where someone is also marked as the primary contact, that flag is applied once per opportunity, as Salesforce requires.

Effect: 437 source rows produced 515 role records. Nothing was discarded.

Transformations applied

Data rarely moves across unchanged. These are the substantive conversions — the cases where a value in Salesforce deliberately differs from what's in Airtable.

Corrections made automatically

- **“Decomissioned” → “Decommissioned”** — a misspelling on 89 application records, corrected on load. Worth fixing at source so it isn't retyped.
- **“Gov?t Employees” → “Gov't Employees”** — 25 opportunity records have a question mark where an apostrophe should be. We confirmed this is genuinely stored that way in Airtable, not a glitch in our export.
- **Impediment categories** — “Issue on their end” and “Relationship Issue” were mapped to the equivalent Salesforce values.

- **Placeholder text removed** — “TBD” entries in web address fields were left blank rather than stored as if they were real links.

Values added to Salesforce so nothing was lost

- **Demographic Served** — Airtable uses 32 categories where Salesforce offered 6. We analysed each category by how recently it was used and added the 24 in active use within the last 18 months. Only 8 genuinely dormant categories were left out, affecting about 28 records rather than the ~400 a simple match would have dropped.
- **Contact roles** — “Day-to-Day POC” and “Senior POC” did not exist in Salesforce. Rather than forcing them into an approximate existing role, both were added.
- **Opportunity type and service level** — several values used throughout Airtable were not selectable on Login.gov opportunities. They were enabled, which also fixed revenue calculations that depend on them.

Fields deliberately not migrated

- **Calculated fields** — revenue totals, completion percentages and similar are recalculated by Salesforce from the underlying data, so writing them across would be overwritten anyway.
- **Market Segment** — set automatically from the related account.
- **Confirmed not needed** — Pilots, Usage Tracker Application Name, Vital Update %, and Issuer Strings, per earlier decisions.

Decisions taken during the load

Where the source data was ambiguous, we stopped rather than guessed. These are the calls that were made, and what each one cost or preserved.

The impediment named “None” was not migrated

One impediment record is named “None”, has no description, and is linked to 465 opportunities — more than five times any real impediment.

Loading it would have asserted that 297 opportunities *are* impeded, and because Salesforce totals blocked revenue per impediment, “None” would have appeared at the top of any “biggest blockers” report with a meaningless multi-million-dollar figure.

Effect: the 267 links from genuine impediments loaded and now show real figures. Reversing this decision takes one setting, not a rebuild.

Contacts without a name were loaded using their email address

Only 491 of 1,599 contacts have a name recorded. Salesforce requires a last name on every contact.

Rather than drop them or invent names, the email address is used as a visible placeholder, so the affected records are easy to find and correct later.

Effect: no contacts lost. About 971 look like real individuals whose name simply wasn't recorded; roughly 116 look like shared mailboxes.

Unmatched accounts were left for human review, not auto-created

169 Airtable account rows match no existing Salesforce account. Creating them automatically would have produced duplicate agency records, since spot-checking showed most are duplicate rows within Airtable rather than genuinely new organisations.

Effect: the largest remaining gap, but a reversible one — each resolved account releases its partner accounts, applications and links automatically.

What we need from you

In rough order of how many records each one unblocks.

HIGHEST IMPACT Resolve the 169 unmatched accounts

For each, tell us whether it duplicates an existing account (and which one is correct), is genuinely new, or is stale. This alone unblocks partner accounts, applications and several hundred links.

DECISION Confirm what the “None” impediment means

If it means “no impediment”, the cleanest fix is to remove those links in Airtable and delete the record.

DECISION Service accounts and shared mailboxes

Roughly 116 contacts are addresses like a help desk rather than people. Should they be contacts at all, or handled another way?

- DATA ENTRY

Names for approximately 971 contacts
The single highest-value cleanup available — ideally captured as first and last name rather than one free-text field.
- DATA ENTRY

28 opportunities have no status
and 16 have no account link, so they cannot be created in Salesforce.
- DECISION

Confirm the record ownership rules
Which account should own records whose Airtable owner has no Salesforce account; whether applications and contacts should take their account's owner; and whether existing accounts' ownership should be left alone. See "Record ownership" above.
- PROVISIONING

Four opportunity leads have no Salesforce account
Covering 285 opportunities between them. Two are already on this list from the partner account review, so resolving them fixes ownership in both places.
- CLARIFY

Does "Technical Emails" mean "Technical POC"?
716 contacts carry this subscription value, which has no Salesforce equivalent. We did not assume the two are the same.
- CLARIFY

Seven opportunities are marked as both blocked by and requesting the same impediment.
We recorded these as blocked, being the more severe reading.

Building the automated loading process

The goal is a repeatable process the team can re-run, not a one-time copy of the data. That distinction matters when reading the table above: a data set can be fully automated and still be mostly unloaded, because it is waiting on source data rather than on engineering. Applications is exactly that — built, tested and proven, but only 65% loaded.

Each data set moves through four stages. *Pull* reads Airtable directly through its API. *Transform* converts it and checks it against what Salesforce will actually accept. *Load* writes it. *Verified* means we confirmed the result in Salesforce rather than trusting the load report.

DATA SET	PULL	TRANSFORM	LOAD	VERIFIED	AUTOMATION NOTES
Accounts	✓	✓	✓	✓	Matches existing accounts rather than creating them, and writes anything it can't match confidently to a review file instead of guessing.

DATA SET	PULL	TRANSFORM	LOAD	VERIFIED	AUTOMATION NOTES
Partner Accounts	✓	✓	✓	✓	Re-runs pick up newly-resolved accounts automatically.
Contacts	✓	✓	✓	✓	Consolidates duplicate people across both source tables as part of the transform.
Opportunities	✓	✓	✓	✓	Required two Salesforce configuration changes, both applied.
Applications	✓	✓	✓	✓	Fully automated; the gap is source data, not the process.
Impediments	✓	✓	✓	✓	Complete apart from two blank source rows.
Application ↔ Contact links	✓	✓	✓	✓	Re-running can't create duplicates, by design.
Impediment ↔ Opportunity links	✓	✓	✓	✓	The "None" exclusion is a setting, not a code change.
Opportunity contact roles	✓	✓	✓	✓	Cannot use the usual matching key, so it compares against existing records instead. Re-running is still safe.
Meetings	✓	—	—	—	Data pulls fine. Deferred rather than blocked — the times don't exist in Airtable to load. See the known issue below.
Notes	✓	—	—	—	Deliberately last — a note must attach to a record that already exists.
Record ownership	✓	✓	—	—	Built once the four decisions were taken; applies at the next load.
Broker application links	✓	✓	—	—	Built. Generated automatically with the applications, loaded as a second step because Salesforce cannot link a record to another in the same batch.

WHAT "AUTOMATED" BUYS YOU

Every step re-reads Airtable and re-checks Salesforce each time it runs. That means the cleanup items on this page don't need an engineer to act on them individually — correcting the source data and re-running is enough to bring the newly-valid records across. Nothing has to be re-keyed by hand, and re-running cannot duplicate what is already loaded.

Record ownership

Requested by the Partnerships team, built, and — as of the 13 August rebuild — **loaded and verified**. This matters more than a reporting column: these objects use owner-based sharing, so the owner decides who can see a record.

The rule agreed in August: where Airtable names an owner and that person has an active Salesforce account, the record is assigned to them; otherwise it goes to a single named fallback owner. No record is ever left

unowned, and no owner is ever invented.

DATA SET	ASSIGNED TO A NAMED PERSON	FALLBACK OWNER	WHERE THE OWNER COMES FROM
Opportunities	471	271	The Pod Opportunity Lead named in Airtable — 11 of 15 leads have an active Salesforce account.
Applications	361	327	Inherited from the partner account's owner.
Contacts	1,548	0	Inherited from the contact's account — but see the finding below.
Impediments	0	39	Airtable records no owner for impediments at all.
Application → Contact links	0	1,878	Airtable records the association, not an owner.

A FINDING THAT NEEDS YOUR DECISION

92% of migrated contacts are owned by a data-loading service account. Of 1,548 contacts, 1,426 are owned by “SystemUser DataLoader”. This is not a fault in the rule — the rule inherits each contact's owner from its account, exactly as agreed, and it worked. The issue is what it inherits *from*: in the production account data, 92% of accounts are owned either by that service account or by one individual.

The result looks like real ownership information but carries almost none. Before go-live it is worth deciding between three options: keep it as is; assign all contacts to the named fallback owner instead, which is at least honest about what is unknown; or inherit only where the account has a genuine individual owner, which would give roughly 8% of contacts a meaningful owner and the rest a clear default.

CAUGHT BY THE REHEARSAL

150 applications were initially rejected by Salesforce because the owner they inherited, while a real and active user, holds a licence type that is **not permitted to own records**. Nothing in the data indicated this — the account looked entirely valid — and the error Salesforce returns names neither the field nor the person.

The loading process now checks licence eligibility as well as whether the account is active, so those applications fall back cleanly instead of failing. This is the clearest example of why the sandbox was emptied and reloaded: the defect was undetectable without doing it.

Proposed rule

Where Airtable records an owner and that person has a Salesforce account, assign the record to them. Where they don't, fall back to a single default owner.

This never leaves a record unowned and never invents an owner. Applied to today's data:

- **Opportunities** — 15 named leads. 11 have Salesforce accounts, covering **541 opportunities**. The other 4 cover 285, which would fall back.
- **Partner Accounts** — 5 of 7 owners resolve, covering 62 records.
- **Applications** — 5 of 7 resolve, covering 847 records.
- **Accounts and Contacts** — Airtable records no owner at all, so these need a decision rather than a rule.

Note: the four Opportunity leads without Salesforce accounts include two people already flagged on this list for the same reason — the same missing accounts block ownership in two places, so provisioning them fixes both.

One thing to know before anyone reports it as a bug: because the fallback owner is written explicitly rather than left blank, re-running a load will push a manually reassigned record back to the fallback owner. This was a deliberate trade — the alternative silently hands thousands of records to whichever engineer happens to run the production load — but it means ownership changes made by hand should be made *after* the final load, not between loads.

Known issue: meetings cannot be migrated as calendar events

This is the one part of the migration that cannot be solved by writing better code, and it needs a decision outside the migration itself.

Airtable records a meeting date, but no start time, end time or duration. Salesforce calendar events require them. There is no version of this transformation that doesn't invent information: a fixed start time would have 1,604 meetings all claiming to begin at 9am, and even an "all-day" event asserts something the source never recorded. Those times would then look exactly like real scheduling history to anyone reading the record later.

Decision: don't invent the times — recover the real ones

Rather than fabricate a schedule, the recommendation is to set up **Einstein Activity Capture**, the Salesforce feature that synchronises users' Google Calendar entries and associates them with Salesforce records. Once real calendar events are present, each Airtable meeting can be matched to the actual event it describes — using the date, who led it, who attended, and the meeting name — and the Airtable summary, agenda and action items attached to that real event.

Effect: the calendar supplies the times, Airtable supplies the meaning, and nothing is fabricated.

Meetings are deferred until this is in place and are not part of the next load.

What this depends on, and what it will not recover

Two things should be understood before committing to this route, because they cap how much meeting history can realistically be recovered:

- **Roughly 3 in 10 meetings can never be matched, regardless of how well it works.** Calendar synchronisation works per person, from that person's own calendar. Only **1,315 of the 1,845** meetings were led by someone who has an active Salesforce user today; the remaining 530 were led by people who have left or were never given an account — one person alone led 182 of them. Those meetings have no calendar to synchronise from. *Giving those people Salesforce accounts, where they are still staff, raises this ceiling and also fixes the record-ownership gaps described above — the same short list of names appears in both.*
- **How far back the calendar history reaches is not yet known.** These meetings span February 2024 to August 2026 — 125 in 2024, 931 in 2025, 548 in 2026. Calendar synchronisation keeps a limited history and may only capture meetings from the day it is switched on. If it does not reach back, older meetings will never appear no matter how good the matching is.

WHAT WE NEED FROM YOU

A decision on whether to pursue Einstein Activity Capture, and who can confirm how far back its calendar history reaches in this organisation. Until that is answered we cannot say how many of the 1,845 meetings are recoverable — only that inventing meeting times is the wrong way to reach a higher number.

We also need a decision on the meetings that will never match: discard them, or keep their summaries and action items as notes attached to the relevant agency or opportunity. The second option preserves the record of what was discussed without pretending a calendar entry existed.

Still to come

Three of the four items on the previous version of this list are now done. What is left is genuinely outstanding rather than merely unloaded.

- **Record ownership** — **DONE** built, loaded and verified. See “Record ownership” above, including one finding that needs a decision.
- **Broker application relationships** — **DONE** 63 of 70 links loaded. The remaining 7 depend on applications still blocked by the account duplicates.
- **Notes** — **DONE** 537 loaded and attached. A further 200 wait on parent records blocked by the same account issue.
- **Meetings** — **DEFERRED** see the known issue above. This is the only data set with no records at all, and it is blocked on a Salesforce configuration decision rather than on migration work.
- **A re-run after cleanup** — every load re-reads Airtable, so fixing the data-quality items above and re-running picks up the newly valid records with no additional engineering. The rebuild demonstrated this: the entire migration was reproduced from an empty sandbox in a single sequence.
- **An undo path** — a rollback script was written alongside this rebuild. It can reverse a single load, and has been tested in preview mode against this one. It is a tidy-up tool, not a substitute for a backup before the production run.

Counts were taken directly from the Dev sandbox (peodv8dvn) on 13 August 2026, after the wipe and reload completed, and from the Airtable export pulled 12 August 2026. Because both systems remain in active use, figures will drift; this report is a snapshot, not a live view. Full field-by-field mapping detail and the complete list of data-quality items are maintained alongside the migration scripts.